```
// ==========================================================================
// 文件: main.dart
// 原始行数: 156 | 保留行数: 60
// ==========================================================================
import 'dart:async';
import 'dart:io';
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'package:provider/provider.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:sqflite_common_ffi/sqflite_ffi.dart';
import 'package:http/http.dart' as http;
import 'ai/tencentcloud/embedded_public_hunyuan_credentials.dart';
import 'ai/tencentcloud/tencent_credentials.dart';
import 'ai/config/auth_service.dart';
import 'ai/config/checkin_service.dart';
import 'ai/config/remote_config_service.dart';
import 'ai/tencentcloud/tencent_api_client.dart';
import 'core/theme/app_theme.dart';
import 'presentation/pages/bookshelf/bookshelf_page.dart';
import 'presentation/providers/ai_model_provider.dart';
import 'presentation/providers/books_provider.dart';
import 'presentation/providers/read_aloud_provider.dart';
import 'presentation/providers/translation_provider.dart';
import 'presentation/providers/qa_stream_provider.dart';
import 'presentation/providers/illustration_provider.dart';

import 'package:flutter/services.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  // Set global system UI mode to edge-to-edge for modern Android experience
  SystemChrome.setEnabledSystemUIMode(SystemUiMode.edgeToEdge);
  SystemChrome.setSystemUIOverlayStyle(const SystemUiOverlayStyle(
    systemNavigationBarColor: Colors.transparent,
    systemNavigationBarDividerColor: Colors.transparent,
    statusBarColor: Colors.transparent,
    systemNavigationBarContrastEnforced:
        false, // Prevent system from enforcing contrast with scrim
    statusBarIconBrightness: Brightness.dark,
    systemNavigationBarIconBrightness: Brightness.dark,
  ));

  if (!kIsWeb && (Platform.isWindows || Platform.isLinux)) {
    sqfliteFfiInit();
    databaseFactory = databaseFactoryFfi;
  }

  final prefs = await SharedPreferences.getInstance();
  bool enabled = prefs.getBool('user_tencent_keys_enabled') ?? false;
  String secretId = (prefs.getString('user_tencent_secret_id') ?? '').trim();
  String secretKey = (prefs.getString('user_tencent_secret_key') ?? '').trim();
```

```
  if (secretId.isEmpty && secretKey.isEmpty) {
    final legacyId = (prefs.getString('dev_tencent_secret_id') ?? '').trim();
    final legacyKey = (prefs.getString('dev_tencent_secret_key') ?? '').trim();
    if (legacyId.isNotEmpty && legacyKey.isNotEmpty) {
      secretId = legacyId;
      secretKey = legacyKey;
      await prefs.setString('user_tencent_secret_id', secretId);
      await prefs.setString('user_tencent_secret_key', secretKey);
      await prefs.setBool('user_tencent_keys_enabled', true);

// ============================================================================
// 文件: data/models/book.dart
// 原始行数: 120 | 保留行数: 120
// ============================================================================
import 'dart:typed_data';

class Book {
  final String id;
  final String title;
  final String author;
  final String coverPath;
  final String filePath;
  final String format;
  final int totalPages;
  final DateTime importDate;
  final int currentPage;
  final double percentage;
  final DateTime? lastRead;
  final int readingChapter;
  final int readingPage;
  final double readingProgress;
  final Uint8List? coverBytes;
  final Uint8List? fileBytes;

  Book({
    required this.id,
    required this.title,
    required this.author,
    this.coverPath = '',
    required this.filePath,
    required this.format,
    this.totalPages = 0,
    required this.importDate,
    this.currentPage = 0,
    this.percentage = 0.0,
    this.lastRead,
    this.readingChapter = 0,
    this.readingPage = 0,
    this.readingProgress = 0.0,
    this.coverBytes,
    this.fileBytes,
  });

  // Convert to Map for Database
  Map<String, dynamic> toMap() {
```

```
  return {
    'id': id,
    'title': title,
    'author': author,
    'cover_path': coverPath,
    'file_path': filePath,
    'format': format,
    'total_pages': totalPages,
    'import_date': importDate.millisecondsSinceEpoch,
    'current_page': currentPage,
    'percentage': percentage,
    'last_read': lastRead?.millisecondsSinceEpoch,
    'reading_chapter': readingChapter,
    'reading_page': readingPage,
    'reading_progress': readingProgress,
  };
}

// Create from Map
factory Book.fromMap(Map<String, dynamic> map) {
  return Book(
    id: map['id'],
    title: map['title'],
    author: map['author'] ?? '',
    coverPath: map['cover_path'] ?? '',
    filePath: map['file_path'] ?? '',
    format: map['format'] ?? 'unknown',
    totalPages: map['total_pages'] ?? 0,
    importDate: DateTime.fromMillisecondsSinceEpoch(map['import_date']),
    currentPage: map['current_page'] ?? 0,
    percentage: map['percentage'] ?? 0.0,
    lastRead: map['last_read'] != null
        ? DateTime.fromMillisecondsSinceEpoch(map['last_read'])
        : null,
    readingChapter: map['reading_chapter'] ?? 0,
    readingPage: map['reading_page'] ?? 0,
    readingProgress: (map['reading_progress'] ?? 0.0) * 1.0,
  );
}

Book copyWith({
  String? id,
  String? title,
  String? author,
  String? coverPath,
  String? filePath,
  String? format,
  int? totalPages,
  DateTime? importDate,
  int? currentPage,
  double? percentage,
  DateTime? lastRead,
  int? readingChapter,
  int? readingPage,
  double? readingProgress,
```

```
    Uint8List? coverBytes,
    Uint8List? fileBytes,
  }) {
    return Book(
      id: id ?? this.id,
      title: title ?? this.title,
      author: author ?? this.author,
      coverPath: coverPath ?? this.coverPath,
      filePath: filePath ?? this.filePath,
      format: format ?? this.format,
      totalPages: totalPages ?? this.totalPages,
      importDate: importDate ?? this.importDate,
      currentPage: currentPage ?? this.currentPage,
      percentage: percentage ?? this.percentage,
      lastRead: lastRead ?? this.lastRead,
      readingChapter: readingChapter ?? this.readingChapter,
      readingPage: readingPage ?? this.readingPage,
      readingProgress: readingProgress ?? this.readingProgress,
      coverBytes: coverBytes ?? this.coverBytes,
      fileBytes: fileBytes ?? this.fileBytes,
    );
  }
}


// ========================================================================
// 文件: data/database/database_helper.dart
// 原始行数: 153 | 保留行数: 153
// ========================================================================
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';
import '../models/book.dart';

class DatabaseHelper {
  static final DatabaseHelper instance = DatabaseHelper._init();
  static Database? _database;

  DatabaseHelper._init();

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB('airread.db');
    return _database!;
  }

  Future<Database> _initDB(String filePath) async {
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, filePath);

    return await openDatabase(path,
        version: 4, onCreate: _createDB, onUpgrade: _upgradeDB);
  }

  Future _createDB(Database db, int version) async {
    const bookTable = '''
```

```
  CREATE TABLE books (
    id TEXT PRIMARY KEY,
    title TEXT NOT NULL,
    author TEXT,
    cover_path TEXT,
    file_path TEXT NOT NULL,
    format TEXT NOT NULL,
    total_pages INTEGER DEFAULT 0,
    import_date INTEGER NOT NULL,
    current_page INTEGER DEFAULT 0,
    percentage REAL DEFAULT 0.0,
    last_read INTEGER,
    reading_chapter INTEGER DEFAULT 0,
    reading_page INTEGER DEFAULT 0,
    reading_progress REAL DEFAULT 0.0
  )
  ''';

  const settingsTable = '''
  CREATE TABLE settings (
    key TEXT PRIMARY KEY,
    value TEXT
  )
  ''';

  await db.execute(bookTable);
  await db.execute(settingsTable);
}

Future _upgradeDB(Database db, int oldVersion, int newVersion) async {
  if (oldVersion < 2) {
    const settingsTable = '''
    CREATE TABLE IF NOT EXISTS settings (
      key TEXT PRIMARY KEY,
      value TEXT
    )
    ''';
    await db.execute(settingsTable);
  }
  if (oldVersion < 3) {
    try {
      await db.execute(
          'ALTER TABLE books ADD COLUMN reading_chapter INTEGER DEFAULT 0');
    } catch (_) {}
    try {
      await db.execute(
          'ALTER TABLE books ADD COLUMN reading_page INTEGER DEFAULT 0');
    } catch (_) {}
  }
  if (oldVersion < 4) {
    try {
      await db.execute(
          'ALTER TABLE books ADD COLUMN reading_progress REAL DEFAULT 0.0');
    } catch (_) {}
  }
```

```
}

Future<void> insertBook(Book book) async {
  final db = await instance.database;
  await db.insert(
    'books',
    book.toMap(),
    conflictAlgorithm: ConflictAlgorithm.replace,
  );
}

Future<List<Book>> getAllBooks() async {
  final db = await instance.database;
  final result = await db.query('books', orderBy: 'import_date DESC');
  return result.map((json) => Book.fromMap(json)).toList();
}

Future<Book?> getBook(String id) async {
  final db = await instance.database;
  final maps = await db.query(
    'books',
    columns: null,
    where: 'id = ?',
    whereArgs: [id],
  );

  if (maps.isNotEmpty) {
    return Book.fromMap(maps.first);
  } else {
    return null;
  }
}

Future<int> updateBook(Book book) async {
  final db = await instance.database;
  return db.update(
    'books',
    book.toMap(),
    where: 'id = ?',
    whereArgs: [book.id],
  );
}

Future<int> deleteBook(String id) async {
  final db = await instance.database;
  return await db.delete(
    'books',
    where: 'id = ?',
    whereArgs: [id],
  );
}

Future<void> saveReadingSettings(double fontSize, double lineHeight) async {
  final db = await instance.database;
  await db.insert(
```

```
      'settings',
      {'key': 'fontSize', 'value': fontSize.toString()},
      conflictAlgorithm: ConflictAlgorithm.replace,
    );
    await db.insert(
      'settings',
      {'key': 'lineHeight', 'value': lineHeight.toString()},
      conflictAlgorithm: ConflictAlgorithm.replace,
    );
  }

  Future<void> close() async {
    final db = await instance.database;
    db.close();
  }
}


// ================================================================================
// 文件: presentation/providers/books_provider.dart
// 原始行数: 333 | 保留行数: 160
// ================================================================================
import 'package:flutter/foundation.dart';
import '../../data/models/book.dart';
import '../../data/database/database_helper.dart';
import '../../data/database/web_database_helper.dart';
import '../../data/services/book_importer.dart';
import 'translation_provider.dart';

import 'package:file_picker/file_picker.dart';

class BooksProvider extends ChangeNotifier {
  List<Book> _books = [];
  bool _isLoading = false;
  bool _isImporting = false;
  String _loadingMessage = '';
  String? _importError;

  // Animation Control
  List<String> _recentlyImportedIds = [];
  int _importBatchId = 0;
  int _booksLoadRequestId = 0;

  final DatabaseHelper _dbHelper = DatabaseHelper.instance;
  final WebDatabaseHelper _webDbHelper = WebDatabaseHelper.instance;
  final BookImporter _importer = BookImporter();

  // Selection Mode
  bool _isSelectionMode = false;
  final Set<String> _selectedBookIds = {};

  List<Book> get books => _books;
  bool get isLoading => _isLoading;
  bool get isImporting => _isImporting;
  String get loadingMessage => _loadingMessage;
```

```
String? get importError => _importError;

bool get isSelectionMode => _isSelectionMode;
Set<String> get selectedBookIds => _selectedBookIds;

List<String> get recentlyImportedIds => _recentlyImportedIds;
int get importBatchId => _importBatchId;

BooksProvider() {
  loadBooks();
}

void _invalidatePendingLoads() {
  _booksLoadRequestId++;
}

void toggleSelectionMode() {
  _isSelectionMode = !_isSelectionMode;
  if (!_isSelectionMode) {
    _selectedBookIds.clear();
  }
  notifyListeners();
}

void toggleBookSelection(String id) {
  if (_selectedBookIds.contains(id)) {
    _selectedBookIds.remove(id);
  } else {
    _selectedBookIds.add(id);
  }
  notifyListeners();
}

void selectAll() {
  if (_selectedBookIds.length == _books.length) {
    _selectedBookIds.clear();
  } else {
    _selectedBookIds.addAll(_books.map((b) => b.id));
  }
  notifyListeners();
}

Future<void> pinSelectedBooks() async {
  if (_selectedBookIds.isEmpty) return;

  _isLoading = true;
  notifyListeners();

  try {
    final now = DateTime.now();
    // Update import date to now (simple "Bump to top" logic)
    final idsToPin = _selectedBookIds.toList();

    for (var id in idsToPin) {
      final index = _books.indexWhere((b) => b.id == id);
```

```
      if (index != -1) {
        final updatedBook = _books[index].copyWith(importDate: now);

        if (kIsWeb) {
          await _webDbHelper
              .insertBook(updatedBook); // Insert replaces on conflict
        } else {
          await _dbHelper
              .insertBook(updatedBook); // Insert replaces on conflict
        }

        _books[index] = updatedBook;
      }
    }

    // Re-sort local list
    _books.sort((a, b) => b.importDate.compareTo(a.importDate));

    // Trigger animation
    _importBatchId++;
    _recentlyImportedIds = [];

    // Exit selection mode
    _isSelectionMode = false;
    _selectedBookIds.clear();
  } catch (e) {
    debugPrint('Error pinning books: $e');
  } finally {
    _isLoading = false;
    notifyListeners();
  }
}

Future<void> deleteSelectedBooks() async {
  if (_selectedBookIds.isEmpty) return;

  _isLoading = true; // Use loading state to prevent interaction
  notifyListeners();

  try {
    final idsToDelete = _selectedBookIds.toList();

    // Update Animation State to trigger re-layout/shift
    _importBatchId++;
    _recentlyImportedIds = []; // No new imports, just layout change

    for (var id in idsToDelete) {
      if (kIsWeb) {
        await _webDbHelper.deleteBook(id);
      } else {
        await _dbHelper.deleteBook(id);
      }
      try {
        await TranslationProvider.removeBookScopedPrefs(id);
      } catch (_) {}
```

```
      }

      _books.removeWhere((b) => idsToDelete.contains(b.id));

      // Exit selection mode
      _isSelectionMode = false;
      _selectedBookIds.clear();

      debugPrint('Deleted ${idsToDelete.length} books');
    } catch (e) {
      debugPrint('Error deleting books: $e');
      // Show error in UI?
    } finally {
      _isLoading = false;
      notifyListeners();
    }
  }

// ================================================================================
// 文件: presentation/pages/bookshelf/bookshelf_page.dart
// 原始行数: 591 | 保留行数: 180
// ================================================================================
import 'dart:async';

import 'package:flutter/material.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/services.dart';
import 'package:file_picker/file_picker.dart';
import 'package:provider/provider.dart';
import '../../providers/books_provider.dart';
import '../../widgets/book_card.dart';
import '../../widgets/air_title.dart';
import '../../widgets/glass_panel.dart';
import '../../../core/theme/app_colors.dart';
import '../../../core/theme/app_tokens.dart';
import '../reader/reader_page.dart';
import '../../../ai/config/app_update_service.dart';
import '../../../data/models/book.dart';

class BookshelfPage extends StatefulWidget {
  const BookshelfPage({super.key});

  @override
  State<BookshelfPage> createState() => _BookshelfPageState();
}

class _BookshelfPageState extends State<BookshelfPage>
    with WidgetsBindingObserver {
  final TextEditingController _searchController = TextEditingController();
  String _searchQuery = '';

  static const _intentChannel = MethodChannel('airread/intent');

  @override
  void initState() {
```

```
  super.initState();
  WidgetsBinding.instance.addObserver(this);
  // System UI mode handled globally in main.dart to ensure consistency
  // 延迟检查应用更新（Android）
  Future.delayed(const Duration(seconds: 3), () {
    if (mounted) AppUpdateService.checkAndPrompt(context);
  });
  // 处理「用灵阅打开」传入的 EPUB 文件（Android）
  if (!kIsWeb) {
    WidgetsBinding.instance.addPostFrameCallback((_) => _checkInitialEpub());
  }
}

@override
void didChangeAppLifecycleState(AppLifecycleState state) {
  if (state != AppLifecycleState.resumed || !mounted) return;
  final booksProvider = Provider.of<BooksProvider>(context, listen: false);
  unawaited(booksProvider.loadBooks());
  if (!kIsWeb) {
    unawaited(_checkInitialEpub());
  }
}

Future<void> _checkInitialEpub() async {
  if (!mounted) return;
  try {
    final path =
        await _intentChannel.invokeMethod<String>('getInitialEpubPath');
    if (path == null || path.isEmpty || !mounted) return;
    final booksProvider = Provider.of<BooksProvider>(context, listen: false);
    final book = await booksProvider.importFromPath(path);
    if (book != null && mounted) {
      Navigator.push(
        context,
        PageRouteBuilder(
          pageBuilder: (context, animation, secondaryAnimation) =>
              ReaderPage(bookId: book.id),
          transitionsBuilder:
              (context, animation, secondaryAnimation, child) {
            const begin = Offset(0.0, 1.0);
            const end = Offset.zero;
            const curve = Curves.easeInOutCubic;
            var tween =
                Tween(begin: begin, end: end).chain(CurveTween(curve: curve));
            return FadeTransition(
              opacity: animation,
              child: SlideTransition(
                  position: animation.drive(tween), child: child),
            );
          },
          transitionDuration: const Duration(milliseconds: 500),
        ),
      );
    }
  } catch (e) {
```

```
    debugPrint('Error handling initial epub: $e');
  }
}

@override
void dispose() {
  WidgetsBinding.instance.removeObserver(this);
  _searchController.dispose();
  super.dispose();
}

Future<void> _handleImport(BuildContext context) async {
  final booksProvider = Provider.of<BooksProvider>(context, listen: false);

  if (kIsWeb) {
    try {
      final result = await FilePicker.platform.pickFiles(
        type: FileType.custom,
        allowedExtensions: ['epub', 'txt'],
        allowMultiple: true,
        withData: true,
      );
      if (result != null) {
        await booksProvider.importWebFiles(result);
      }
    } catch (e) {
      debugPrint('Error picking files on web: $e');
    }
  } else {
    await booksProvider.importBooks();
  }
}

void _onSearchChanged(String value) {
  setState(() {
    _searchQuery = value.toLowerCase();
  });
}

List<Book> _filterBooks(List<Book> books) {
  if (_searchQuery.isEmpty) return books;
  return books.where((book) {
    return book.title.toLowerCase().contains(_searchQuery) ||
        book.author.toLowerCase().contains(_searchQuery);
  }).toList();
}

@override
Widget build(BuildContext context) {
  final booksProvider = Provider.of<BooksProvider>(context);
  final theme = Theme.of(context);
  final scheme = theme.colorScheme;
  final isDark = theme.brightness == Brightness.dark;
  // Filter books based on search query
  final displayBooks = _filterBooks(booksProvider.books);
```

```
    return Scaffold(
      extendBodyBehindAppBar: true,
      resizeToAvoidBottomInset:
          false, // Prevent keyboard/system UI from resizing layout
      bottomNavigationBar: SafeArea(
        top: false,
        child: Padding(
          padding: const EdgeInsets.symmetric(vertical: 8),
          child: SizedBox(
            width: double.infinity,
            child: Text(
           kIsWeb ? '浙ICP备2026011869号-1' : '浙ICP备2026011869号-2A',
              textAlign: TextAlign.center,
              style: TextStyle(
                fontSize: 11,
                color: scheme.onSurface.withOpacityCompat(0.35),
              ),
            ),
          ),
        ),
      ),
      appBar: AppBar(
        systemOverlayStyle: SystemUiOverlayStyle(
          statusBarColor: Colors.transparent,
          systemNavigationBarColor: Colors.transparent,
          statusBarIconBrightness: isDark ? Brightness.light : Brightness.dark,
          systemNavigationBarIconBrightness:
              isDark ? Brightness.light : Brightness.dark,
          systemNavigationBarContrastEnforced: false,
        ),
        backgroundColor: Colors.transparent,
        elevation: 0,
        centerTitle: true,
        title: const AirTitle(),
        automaticallyImplyLeading: false,
      ),
```

```
// ============================================================================
// 文件: presentation/pages/reader/reader_page.dart
// 原始行数: 7759 | 保留行数: 280
// ============================================================================
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter/foundation.dart';
import 'package:provider/provider.dart';
import 'package:epubx/epubx.dart';
import 'dart:async';
import 'dart:collection';
import 'dart:convert';
import 'dart:io';
import 'package:path_provider/path_provider.dart';
import 'package:audioplayers/audioplayers.dart';
import 'package:shared_preferences/shared_preferences.dart';
import '../../../core/theme/app_colors.dart';
```

```
import '../../../core/theme/app_tokens.dart';
import '../../widgets/ai_hud.dart';
import '../../widgets/glass_panel.dart';
import '../../providers/books_provider.dart';
import '../../providers/ai_model_provider.dart';
import '../../providers/read_aloud_provider.dart';
import '../../providers/translation_provider.dart';
import '../../../data/services/book_parser.dart';
import '../../../data/models/book.dart';
import 'widgets/reader_paragraph.dart';
import '../../../ai/translation/translation_types.dart';
import '../../../ai/tencentcloud/embedded_public_hunyuan_credentials.dart';
import '../../../ai/tencent_tts/tencent_tts_client.dart';
import 'tts_web_speech.dart'
    if (dart.library.js_interop) 'tts_web_speech_web.dart';

/// 解码 HTML 实体（如 &#45556;、 ），修复灵译等翻译 EPUB 中的乱码
String _decodeHtmlEntities(String html) {
  String s = html;
  // 十进制 &#DDDDD; 或 &#DDDDD（无分号）
  s = s.replaceAllMapped(RegExp(r'&#(\d{1,7});?'), (m) {
    final n = int.tryParse(m[1]!);
    if (n == null || n > 0x10FFFF) return m[0]!;
    return String.fromCharCode(n);
  });
  // 十六进制 &#xHHHH; 或 &#xHHHH
  s = s.replaceAllMapped(RegExp(r'&#x([0-9a-fA-F]{1,6});?'), (m) {
    final n = int.tryParse(m[1]!, radix: 16);
    if (n == null) return m[0]!;
    if (n > 0x10FFFF) return m[0]!;
    return String.fromCharCode(n);
  });
  // 常见命名实体
  const named = {
    '&amp;': '&', '&lt;': '<', '&gt;': '>', '&quot;': '"', '&apos;': "'",
    ' ': ' ', ' ': ' ', ' ': ' ', ' ': ' ', ' ': ' ',
  };
  for (final e in named.entries) {
    s = s.replaceAll(e.key, e.value);
  }
  return s;
}

class _MeasureSize extends StatefulWidget {
  final Widget child;
  final ValueChanged<Size> onChange;

  const _MeasureSize({
    required this.onChange,
    required this.child,
  });

  @override
  _MeasureSizeState createState() => _MeasureSizeState();
}
```

```
class _MeasureSizeState extends State<_MeasureSize> {
  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(
      builder: (context, constraints) {
        WidgetsBinding.instance.addPostFrameCallback((_) {
          if (context.mounted) {
            final size = context.size;
            if (size != null) {
              widget.onChange(size);
            }
          }
        });
        return widget.child;
      },
    );
  }
}

abstract class _ReaderChapter {
  String? get title;
  Future<String> readHtmlContent();
}

class _EpubReaderChapter implements _ReaderChapter {
  final EpubChapterRef ref;
  final String? _titleOverride;
  final bool hasSubChapters;
  final int? parentIndex;
  _EpubReaderChapter(this.ref,
      {String? titleOverride, this.hasSubChapters = false, this.parentIndex})
      : _titleOverride = titleOverride;

  @override
  String? get title => _titleOverride ?? ref.Title;

  @override
  Future<String> readHtmlContent() => ref.readHtmlContent();
}

class _TextReaderChapter implements _ReaderChapter {
  final String _title;
  /// Only holds the chapter's own text (substring), not the entire book.
  final String _chapterText;

  _TextReaderChapter({
    required String title,
    required String sourceText,
    required int start,
    required int end,
  })  : _title = title,
        _chapterText = sourceText.substring(
          start.clamp(0, sourceText.length),
          end.clamp(start.clamp(0, sourceText.length), sourceText.length),
```

```
      );

@override
String? get title => _title;

@override
Future<String> readHtmlContent() async => '';

Future<String> readPlainText() async {
  final buffer = StringBuffer();
  // Always write the chapter title at the beginning
 final titleText = _title.trim().isEmpty ? '正文' : _title.trim();
  buffer.write(titleText);

  final int start = 0;
  final int end = _chapterText.length;
  if (start >= end) return buffer.toString();
  buffer.write('\n\n');

  final paraBuffer = StringBuffer();
  String? prevLineInPara;
  int processedLines = 0;

  bool isFirstPara = true;
  void flushPara() {
    if (paraBuffer.isEmpty) return;
    if (isFirstPara) {
      isFirstPara = false;
    } else {
      buffer.write('\n\n');
    }
    buffer.write(paraBuffer.toString());
    paraBuffer.clear();
    prevLineInPara = null;
  }

  bool looksLikeParagraphStart(String rawLine, String t) {
    if (rawLine.startsWith('\u3000\u3000')) return true;
    if (rawLine.startsWith('  ')) return true;
    if (rawLine.startsWith('\t')) return true;
    if (RegExp(r'^[-*•]\s+').hasMatch(t)) return true;
    if (RegExp(r'^\d{1,3}[\.■]\s*').hasMatch(t)) return true;
   if (RegExp(r'^[一二三四五六七八九十]{1,3}[、\.]\s*').hasMatch(t)) {
      return true;
    }
    return false;
  }

  bool endsSentence(String s) {
    if (s.isEmpty) return false;
    final last = s.codeUnitAt(s.length - 1);
    if (last == 0x3002) return true;
    if (last == 0xFF01) return true;
    if (last == 0xFF1F) return true;
    if (last == 0x002E) return true;
```

```
  if (last == 0x003F) return true;
  if (last == 0x0021) return true;
  if (s.endsWith('……')) return true;
  if (s.endsWith('...')) return true;
  return false;
}

int sampleNonEmpty = 0;
int sampleBlank = 0;
int sampleIndent = 0;
int sampleLenSum = 0;
int sampleLines = 0;
int j = start;
while (j < end && sampleLines < 200) {
  int lineStart = j;
  int lineEnd = j;
  while (lineEnd < end) {
    final int cu = _chapterText.codeUnitAt(lineEnd);
    if (cu == 10 || cu == 13) break;
    lineEnd++;
  }
  int next = lineEnd;
  if (next < end) {
    final int cu = _chapterText.codeUnitAt(next);
    if (cu == 13) {
      next++;
      if (next < end && _chapterText.codeUnitAt(next) == 10) {
        next++;
      }
    } else if (cu == 10) {
      next++;
    }
  }
  final rawLine = _chapterText.substring(lineStart, lineEnd);
  final t = rawLine.trim();
  if (t.isEmpty) {
    sampleBlank++;
  } else {
    sampleNonEmpty++;
    sampleLenSum += t.length;
    if (looksLikeParagraphStart(rawLine, t)) sampleIndent++;
  }
  sampleLines++;
  j = next;
}

final bool hasBlankLines = sampleBlank >= 2;
final double avgLen =
    sampleNonEmpty == 0 ? 0 : (sampleLenSum / sampleNonEmpty);
final double indentRatio =
    sampleNonEmpty == 0 ? 0 : (sampleIndent / sampleNonEmpty);

final bool splitEachLine =
    !hasBlankLines && indentRatio < 0.12 && avgLen > 0 && avgLen <= 40;
final bool wrapByPunctuation =
```

```
        !hasBlankLines && !splitEachLine && avgLen >= 55 && indentRatio < 0.08;

    int i = start;
    while (i < end) {
      int lineStart = i;
      int lineEnd = i;
      while (lineEnd < end) {
        final int cu = _chapterText.codeUnitAt(lineEnd);
        if (cu == 10 || cu == 13) break;
        lineEnd++;
      }

      int next = lineEnd;
      if (next < end) {
        final int cu = _chapterText.codeUnitAt(next);
        if (cu == 13) {
          next++;
          if (next < end && _chapterText.codeUnitAt(next) == 10) {
            next++;
          }
        } else if (cu == 10) {
          next++;
        }
      }

      final rawLine = _chapterText.substring(lineStart, lineEnd);
      final t = rawLine.trim();

      if (t.isEmpty) {
        flushPara();
      } else {
        if (paraBuffer.isNotEmpty && looksLikeParagraphStart(rawLine, t)) {
          flushPara();
        }
        if (splitEachLine) {
          if (paraBuffer.isNotEmpty) {
            flushPara();
          }
          paraBuffer.write(t);
          prevLineInPara = t;
          flushPara();
        } else {
          if (paraBuffer.isNotEmpty) {
            final glue =
                (prevLineInPara != null && _isCjk(prevLineInPara!) && _isCjk(t))
                    ? ''
                    : ' ';

// ================================================================================
// 文件: presentation/pages/reader/widgets/reader_paragraph.dart
// 原始行数: 14 | 保留行数: 14
// ================================================================================
class ReaderParagraph {
  final int index;
  final int start;
```

```
  final int end;
  final String text;

  const ReaderParagraph({
    required this.index,
    required this.start,
    required this.end,
    required this.text,
  });
}


// ================================================================================
// 文件: ai/translation/translation_types.dart
// 原始行数: 46 | 保留行数: 46
// ================================================================================
import 'package:flutter/foundation.dart';

enum TranslationDisplayMode {
  translationOnly,
  bilingual,
}

@immutable
class TranslationConfig {
  /// Empty means auto-detect (if supported by engine).
  final String sourceLang;

  /// Required.
  final String targetLang;
  final TranslationDisplayMode displayMode;

  const TranslationConfig({
    required this.sourceLang,
    required this.targetLang,
    required this.displayMode,
  });

  TranslationConfig copyWith({
    String? sourceLang,
    String? targetLang,
    TranslationDisplayMode? displayMode,
  }) {
    return TranslationConfig(
      sourceLang: sourceLang ?? this.sourceLang,
      targetLang: targetLang ?? this.targetLang,
      displayMode: displayMode ?? this.displayMode,
    );
  }
}

@immutable
class TranslationResult {
  final String text;
  final bool fromCache;
```

```
  const TranslationResult({
    required this.text,
    required this.fromCache,
  });
}


// ========================================================================
// 文件: ai/translation/translation_service.dart
// 原始行数: 361 | 保留行数: 160
// ========================================================================
import 'dart:async';
import 'dart:collection';

import 'engines/translation_engine.dart';

import 'translation_cache.dart';
import 'translation_queue.dart';
import 'translation_types.dart';

enum TranslationBackend {
  local,
  online,
}

class TranslationService {
  final TranslationCache cache;
  final void Function(String message, Object? error, StackTrace? st)? logger;

  final TranslationTaskQueue _localQueue =
      TranslationTaskQueue(maxConcurrent: 1);
  final TranslationTaskQueue _onlineQueue =
      TranslationTaskQueue(maxConcurrent: 3);

  final Map<String, Future<String>> _inFlight = {};

  final TranslationEngine engine;
  final TranslationBackend backend;

  /// AI context cache: keep last 3 source paragraphs per (targetLang).

  final Map<String, ListQueue<String>> _aiContextSources = {};

  TranslationService({
    required this.cache,
    this.logger,
    required this.engine,
    required this.backend,
  });

  Duration get _translateTimeout {
    switch (backend) {
      case TranslationBackend.local:
        return const Duration(seconds: 70);
```

```
    case TranslationBackend.online:
      return const Duration(seconds: 45);
  }
}

TranslationTaskQueue get _queue {
  switch (backend) {
    case TranslationBackend.local:
      return _localQueue;
    case TranslationBackend.online:
      return _onlineQueue;
  }
}

String normalizeParagraphText(String text) {
  return text.trim();
}

String buildCacheKey({
  required TranslationConfig config,
  required String paragraphText,
}) {
  final normalized = normalizeParagraphText(paragraphText);
  return cache.buildKey(
    engineId: engine.id,
    sourceLang: config.sourceLang,
    targetLang: config.targetLang,
    text: normalized,
  );
}

Future<String> translateParagraph({
  required TranslationConfig config,
  required String paragraphText,
}) async {
  final normalized = normalizeParagraphText(paragraphText);
  if (!_shouldTranslateSource(normalized)) {
    return paragraphText;
  }

  final sourceKey = _langKey(config.sourceLang);
  final targetKey = _langKey(config.targetLang);
  if (targetKey.isEmpty) {
    return paragraphText;
  }
  if (sourceKey.isNotEmpty && sourceKey == targetKey) {
    return paragraphText;
  }
  if (sourceKey.isEmpty) {
    final guessed = _guessLangFromText(normalized);
    if (guessed.isNotEmpty && guessed == targetKey) {
      return paragraphText;
    }
  }
```

```
  final cacheKey = cache.buildKey(
    engineId: engine.id,
    sourceLang: config.sourceLang,
    targetLang: config.targetLang,
    text: normalized,
  );

  final cached = await cache.get(cacheKey);
  if (cached != null) {
    return cached;
  }

  final existing = _inFlight[cacheKey];
  if (existing != null) return existing;

  final future = _queue.submit(() async {
    final translated = await _withRetry(() async {
      final context = _getAiContext(config);
      final out = await engine
          .translate(
            text: normalized,
            sourceLang: config.sourceLang,
            targetLang: config.targetLang,
            contextSources: context,
          )
          .timeout(_translateTimeout);
      final cleaned = out.trim();
      if (cleaned.isEmpty) {
        throw StateError('empty translation');
      }
      if (_looksLikeBadTranslation(cleaned, normalized, config)) {
        throw StateError('bad translation');
      }
      return cleaned;
    });

    await cache.set(cacheKey, translated);

    _onAiTranslated(
      config: config,
      sourceParagraph: normalized,
      translatedParagraph: translated,
    );

    return translated;
  });

  _inFlight[cacheKey] = future;
  return future.whenComplete(() {
    _inFlight.remove(cacheKey);
  });
}

bool _shouldTranslateSource(String text) {
  final s = text.trim();
```

```
    if (s.isEmpty) return false;
    for (final r in s.runes) {
      if (r >= 0x30 && r <= 0x39) return true;
      if ((r >= 0x41 && r <= 0x5A) || (r >= 0x61 && r <= 0x7A)) return true;
      if (r >= 0x00C0 && r <= 0x024F) return true;
      if (r >= 0x0370 && r <= 0x03FF) return true;
      if (r >= 0x0400 && r <= 0x04FF) return true;

// ============================================================================
// 文件: presentation/providers/translation_provider.dart
// 原始行数: 1561 | 保留行数: 200
// ============================================================================
import 'dart:async';
import 'dart:convert';

import 'package:flutter/foundation.dart';
import 'package:flutter/services.dart';
import 'package:http/http.dart' as http;

import 'package:shared_preferences/shared_preferences.dart';
import '../../ai/hunyuan/hunyuan_translation_engine.dart';
import '../../ai/config/auth_service.dart';
import '../../ai/tencentcloud/embedded_public_hunyuan_credentials.dart';
import '../../ai/tencentcloud/tencent_api_client.dart';
import '../../ai/tencentcloud/tencent_cloud_exception.dart';
import '../../ai/tencentcloud/tmt_translation_engine.dart';
import '../../ai/translation/engines/translation_engine.dart';
import '../../ai/translation/translation_cache.dart';
import '../../ai/translation/translation_service.dart';
import '../../ai/translation/translation_types.dart';
import 'ai_model_provider.dart';

enum TranslationMode {
  machine,
  bigModel,
}

enum ReadAloudEngine {
  local,
  online,
}

class AzureTranslationEngine implements TranslationEngine {
  static const String _defaultEndpoint = String.fromEnvironment(
    'AIRREAD_AZURE_TRANSLATOR_ENDPOINT',
    defaultValue: 'https://api-edge.cognitive.microsofttranslator.com',
  );
  static const String _defaultKey =
      String.fromEnvironment('AIRREAD_AZURE_TRANSLATOR_KEY', defaultValue: '');
  static const String _defaultRegion = String.fromEnvironment(
    'AIRREAD_AZURE_TRANSLATOR_REGION',
    defaultValue: '',
  );

  final http.Client _client;
```

```
final String endpoint;
final String key;
final String region;

AzureTranslationEngine({
  http.Client? client,
  String? endpoint,
  String? key,
  String? region,
})  : _client = client ?? http.Client(),
      endpoint = (endpoint ?? _defaultEndpoint).trim(),
      key = (key ?? _defaultKey).trim(),
      region = (region ?? _defaultRegion).trim();

static bool get isConfigured => _defaultEndpoint.trim().isNotEmpty;

bool get isUsable => endpoint.isNotEmpty;

@override
String get id => 'azure_translator';

@override
Future<String> translate({
  required String text,
  required String sourceLang,
  required String targetLang,
  required List<String> contextSources,
}) async {
  final normalized = text.trim();
  if (normalized.isEmpty) return '';
  if (!isUsable) {
    throw StateError('Azure translator not configured');
  }

  final from = _normalizeAzureLang(sourceLang, allowEmpty: true);
  final to = _normalizeAzureLang(targetLang);
  if (to.isEmpty) {
    throw StateError('Azure target language missing');
  }

  final uri = _buildUri(from: from, to: to);
  final headers = <String, String>{
    'Content-Type': 'application/json; charset=utf-8',
  };
  if (key.isNotEmpty) {
    headers['Ocp-Apim-Subscription-Key'] = key;
    if (region.isNotEmpty) {
      headers['Ocp-Apim-Subscription-Region'] = region;
    }
  } else {
    final token = await _getEdgeToken();
    headers['Authorization'] = 'Bearer $token';
    headers['User-Agent'] =
        'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/1
  }
```

```
final body = jsonEncode([
  {'Text': normalized}
]);

debugPrint(
  'AzureTranslator request to=$to from=${from.isEmpty ? 'auto' : from} endpoint=${uri.host}',
);
final resp = await _client
    .post(uri, headers: headers, body: body)
    .timeout(const Duration(seconds: 18));

if (resp.statusCode < 200 || resp.statusCode >= 300) {
  debugPrint('AzureTranslator error status=${resp.statusCode}');
  throw StateError('Azure translate HTTP ${resp.statusCode}: ${resp.body}');
}

debugPrint(
    'AzureTranslator response status=${resp.statusCode} bytes=${resp.bodyBytes.length}');
// Log the full body for debugging purposes
debugPrint('AzureTranslator response body: ${utf8.decode(resp.bodyBytes)}');

final decoded = jsonDecode(utf8.decode(resp.bodyBytes));
if (decoded is! List || decoded.isEmpty) {
  throw StateError('Azure translate response empty');
}
final first = decoded.first;
if (first is! Map) {
  throw StateError('Azure translate response invalid');
}
final detectedLanguage = first['detectedLanguage'];
final detected = detectedLanguage is Map
    ? (detectedLanguage['language']?.toString() ?? '')
    : '';
final detectedNormalized = detected.trim().isEmpty
    ? ''
    : _normalizeAzureLang(detected, allowEmpty: true);
final translations = first['translations'];
if (translations is List && translations.isNotEmpty) {
  final item = translations.first;
  if (item is Map) {
    final out = item['text']?.toString() ?? '';
    if (out.trim().isEmpty) {
      throw StateError('Azure translate result empty');
    }
    // Check for echo translation (source == target) which often indicates failure
    if (out.trim() == normalized && normalized.length > 5) {
      if (detectedNormalized.isNotEmpty &&
          detectedNormalized.toLowerCase() == to.toLowerCase()) {
        return normalized;
      }
      throw StateError('Azure translate result equals source (echo)');
    }
    return out;
  }
```

```
    }
    throw StateError('Azure translate result not found');
  }

  @override
  Future<List<String>> translateBatch({
    required List<String> texts,
    required String sourceLang,
    required String targetLang,
  }) async {
    final out = <String>[];
    for (final t in texts) {
      out.add(await translate(
        text: t,
        sourceLang: sourceLang,
        targetLang: targetLang,
        contextSources: const [],
      ));
    }
    return out;
  }

  Uri _buildUri({required String from, required String to}) {
    final base = Uri.parse(endpoint);
    final basePath = base.path.trim().isEmpty ? '/' : base.path;
    final resolvedPath =
        basePath.endsWith('/') ? '${basePath}translate' : '$basePath/translate';
    final params = <String, String>{
      'api-version': '3.0',
      'to': to,
    };
    if (from.isNotEmpty) {
      params['from'] = from;
    }
    return base.replace(path: resolvedPath, queryParameters: params);
  }

  Future<String> _getEdgeToken() async {
    final now = DateTime.now();
    final cached = _edgeTokenCache;
    if (cached != null && cached.expiresAt.isAfter(now)) {
      return cached.token;
    }
    final uri = Uri.parse('https://edge.microsoft.com/translate/auth');
    final resp = await _client.get(uri, headers: const {
      'User-Agent':
          'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/1

// ============================================================================
// 文件: presentation/pages/reader/widgets/translation_sheet.dart
// 原始行数: 258 | 保留行数: 100
// ============================================================================
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
```

```
import '../../../../core/theme/app_colors.dart';
import '../../../../core/theme/app_tokens.dart';
import '../../../widgets/glass_panel.dart';
import '../../../providers/translation_provider.dart';
import '../../../../ai/translation/translation_types.dart';

/// Translation settings sheet.
///
/// Note: In this app the "apply translation to reader" switch is controlled from
/// the AI companion main panel. This sheet focuses on configuration.
class TranslationSheet extends StatelessWidget {
  final Color bgColor;
  final Color textColor;

  const TranslationSheet({
    super.key,
    required this.bgColor,
    required this.textColor,
  });

  static const _langs = <String, String>{
    '': '自动',
    'zh-Hans': '中文',
    'en': '英语',
    'ja': '日语',
    'ko': '韩语',
    'fr': '法语',
    'de': '德语',
    'es': '西班牙语',
    'ru': '俄语',
  };

  @override
  Widget build(BuildContext context) {
    final provider = context.watch<TranslationProvider>();
    final cfg = provider.config;

    final panelBg = bgColor;
    final panelText = textColor;

    return GlassPanel.sheet(
      surfaceColor: panelBg,
      opacity: AppTokens.glassOpacityDense,
      child: SafeArea(
        top: false,
        child: SizedBox(
          height: MediaQuery.of(context).size.height * 0.72,
          child: Padding(
            padding: const EdgeInsets.fromLTRB(24, 16, 24, 24),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Row(
                  children: [
                    const Icon(Icons.translate, color: AppColors.techBlue),
```

```
                    const SizedBox(width: 8),
                    Text(
                  '翻译设置',
                      style: TextStyle(
                        fontSize: 16,
                        fontWeight: FontWeight.bold,
                        color: panelText,
                      ),
                    ),
                    const Spacer(),
                    IconButton(
                      icon:
                          Icon(Icons.close, color: panelText.withOpacityCompat(0.7)),
                      onPressed: () => Navigator.pop(context),
                    ),
                  ],
                ),
                const SizedBox(height: 12),
                _buildCard(
                  panelBg: panelBg,
                  panelText: panelText,
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Row(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: [
                          Expanded(
                            child: _dropdown(
                           label: '源语言（可选）',
                                value: cfg.sourceLang,
                                items: _langs,
                                onChanged: (v) => provider.setSourceLang(v ?? ''),
                                textColor: panelText,
                                dropdownColor: panelBg,
                              ),
                           ),
                           const SizedBox(width: 12),
                           Expanded(
                             child: _dropdown(
                           label: '目标语言（必选）',
                                value: cfg.targetLang,

// ========================================================================
// 文件: ai/tencent_tts/tencent_tts_client.dart
// 原始行数: 298 | 保留行数: 160
// ========================================================================
import 'dart:async';
import 'dart:convert';
import 'dart:math';
import 'dart:typed_data';

import 'package:crypto/crypto.dart';

import '../tencentcloud/tencent_api_client.dart';
```

```
import '../tencentcloud/tencent_credentials.dart';
import '../tencentcloud/embedded_public_hunyuan_credentials.dart';
import 'tts_ws.dart';

class TencentTtsResult {
  final String audioBase64;
  final String requestId;

  const TencentTtsResult({
    required this.audioBase64,
    required this.requestId,
  });
}

class TencentTtsClient {
  static const String _host = 'tts.tencentcloudapi.com';
  static const String _service = 'tts';
  static const String _version = '2019-08-23';

  static const String _streamHost = 'tts.cloud.tencent.com';
  static const String _streamPath = '/stream_wsv2';

  final TencentApiClient _api;
  final TencentCredentials _credentials;

  TencentTtsClient({
    TencentApiClient? api,
    required TencentCredentials credentials,
  })  : _api = api ?? TencentApiClient(),
        _credentials = credentials;

  Future<TencentTtsResult> textToVoice({
    required String text,
    String codec = 'mp3',
    int? voiceType,
    double? speed,
  }) async {
    final sessionId = _randomSessionId();
    final payload = <String, dynamic>{
      'Text': text,
      'SessionId': sessionId,
      'Codec': codec,
    };
    if (voiceType != null) payload['VoiceType'] = voiceType;
    if (speed != null) payload['Speed'] = speed;

    final resp = await _api.postJson(
      host: _host,
      service: _service,
      action: 'TextToVoice',
      version: _version,
      secretId: _credentials.secretId,
      secretKey: _credentials.secretKey,
      useProxy: !usingPersonalTencentKeys(),
      payload: payload,
```

```
    timeout: const Duration(seconds: 30),
  );

  return TencentTtsResult(
    audioBase64: resp['Audio']?.toString() ?? '',
    requestId: resp['RequestId']?.toString() ?? '',
  );
}

Future<Uint8List> streamTextToVoiceBytes({
  required String text,
  String codec = 'mp3',
  int? voiceType,
  double? speed,
  int sampleRate = 16000,
  bool enableSubtitle = false,
  Duration timeout = const Duration(seconds: 90),
}) async {
  if (!usingPersonalTencentKeys()) {
    final res = await textToVoice(
      text: text,
      codec: codec,
      voiceType: voiceType,
      speed: speed,
    );
    final audioBase64 = res.audioBase64.trim();
    if (audioBase64.isEmpty) {
     throw StateError('腾讯语音合成返回空音频');
    }
    return base64Decode(audioBase64);
  }
  final appIdText = _credentials.appId.trim();
  if (appIdText.isEmpty) {
   throw const FormatException('缺少 AppId，无法调用流式语音合成接口');
  }
  final appId = int.tryParse(appIdText);
  if (appId == null) {
   throw const FormatException('AppId 必须为整数');
  }
  final secretId = _credentials.secretId.trim();
  final secretKey = _credentials.secretKey.trim();
  if (secretId.isEmpty || secretKey.isEmpty) {
   throw const FormatException('缺少 SecretId/SecretKey，无法调用流式语音合成接口');
  }

  final sessionId = _randomSessionId();
  final url = _buildStreamWsv2Url(
    appId: appId,
    secretId: secretId,
    secretKey: secretKey,
    sessionId: sessionId,
    codec: codec,
    voiceType: voiceType,
    speed: speed,
    sampleRate: sampleRate,
```

```
      enableSubtitle: enableSubtitle,
    );

    final audio = BytesBuilder(copy: false);
    final ready = Completer<void>();
    final finished = Completer<void>();

    TtsWebSocket? ws;
    Timer? watchdog;

    void completeError(Object error, [StackTrace? st]) {
      if (!ready.isCompleted) {
        ready.completeError(error, st);
      }
      if (!finished.isCompleted) {
        finished.completeError(error, st);
      }
    }

    try {
      ws = await connectTtsWebSocket(url);

      watchdog = Timer(timeout, () {
       completeError(TimeoutException('语音合成超时'));
        ws?.close();
      });

      ws.stream.listen(
        (data) {
          if (finished.isCompleted) return;

          if (data is String) {
            try {
              final decoded = jsonDecode(data);
              if (decoded is! Map) return;

              final code = decoded['code'];
              if (code is num && code.toInt() != 0) {
                final msg = decoded['message']?.toString() ?? 'Unknown error';
               completeError(StateError('腾讯语音合成错误($code): $msg'));
                ws?.close();
                return;

// ============================================================================
// 文件: presentation/providers/read_aloud_provider.dart
// 原始行数: 1419 | 保留行数: 200
// ============================================================================
import 'dart:async';
import 'dart:convert';
import 'dart:io';

import 'package:audioplayers/audioplayers.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/services.dart';
import 'package:path_provider/path_provider.dart';
```

```
import 'package:shared_preferences/shared_preferences.dart';

import '../../ai/tencent_tts/tencent_tts_client.dart';
import '../../ai/tencentcloud/tencent_cloud_exception.dart';
import '../../ai/tencentcloud/embedded_public_hunyuan_credentials.dart';
import '../../ai/translation/translation_types.dart';
import '../pages/reader/tts_web_speech.dart';
import '../pages/reader/widgets/reader_paragraph.dart';
import 'translation_provider.dart';

class ReadAloudPosition {
  final String bookId;
  final int chapterIndex;
  final int paragraphIndex;
  final int chunkIndexInParagraph;
  final int highlightOffsetInParagraph;
  final int chapterTextOffset;
  final String highlightText;

  const ReadAloudPosition({
    required this.bookId,
    required this.chapterIndex,
    required this.paragraphIndex,
    required this.chunkIndexInParagraph,
    required this.highlightOffsetInParagraph,
    required this.chapterTextOffset,
    required this.highlightText,
  });

  Map<String, dynamic> toJson() => {
        'bookId': bookId,
        'chapterIndex': chapterIndex,
        'paragraphIndex': paragraphIndex,
        'chunkIndexInParagraph': chunkIndexInParagraph,
        'highlightOffsetInParagraph': highlightOffsetInParagraph,
        'chapterTextOffset': chapterTextOffset,
        'highlightText': highlightText,
      };

  static ReadAloudPosition? tryParse(dynamic value) {
    if (value is! Map) return null;
    final bookId = (value['bookId'] ?? '').toString();
    final chapterIndex = _asInt(value['chapterIndex']);
    final paragraphIndex = _asInt(value['paragraphIndex']);
    final chunkIndexInParagraph = _asInt(value['chunkIndexInParagraph']);
    final highlightOffsetInParagraph =
        _asInt(value['highlightOffsetInParagraph']) ?? 0;
    final chapterTextOffset = _asInt(value['chapterTextOffset']) ?? 0;
    final highlightText = (value['highlightText'] ?? '').toString();
    if (bookId.isEmpty ||
        chapterIndex == null ||
        paragraphIndex == null ||
        chunkIndexInParagraph == null) {
      return null;
    }
```

```
    return ReadAloudPosition(
      bookId: bookId,
      chapterIndex: chapterIndex,
      paragraphIndex: paragraphIndex,
      chunkIndexInParagraph: chunkIndexInParagraph,
      highlightOffsetInParagraph: highlightOffsetInParagraph,
      chapterTextOffset: chapterTextOffset,
      highlightText: highlightText,
    );
  }

  static int? _asInt(dynamic v) {
    return switch (v) {
      int x => x,
      num x => x.toInt(),
      String x => int.tryParse(x),
      _ => null,
    };
  }
}

class ReadAloudProvider extends ChangeNotifier {
  static const MethodChannel _localTtsChannel =
      MethodChannel('airread/local_tts');
  static const EventChannel _localTtsEvents =
      EventChannel('airread/local_tts_events');

  TranslationProvider? _tp;
  ReadAloudEngine? _lastEngine;
  int? _lastVoiceType;
  double? _lastTtsSpeed;
  double? _lastLocalTtsSpeed;
  bool? _lastApplyToReader;
  TranslationDisplayMode? _lastDisplayMode;
  String? _lastSourceLang;
  String? _lastTargetLang;
  int? _lastCacheRevision;
  bool? _lastAiReadAloudEnabled;
  int? _lastPointsBalance;
  bool? _lastUsingPersonalTencentKeys;
  bool _queueDirty = false;
  SharedPreferences? _prefs;
  StreamSubscription<dynamic>? _localTtsSub;
  StreamSubscription<void>? _playerCompleteSub;

  AudioPlayer? _playerInstance;
  AudioPlayer get _player {
    if (_playerInstance == null) {
      _playerInstance = AudioPlayer();
      _playerCompleteSub?.cancel();
      _playerCompleteSub = _playerInstance!.onPlayerComplete.listen((_) {
        _onOnlineChunkDone();
      });
    }
    return _playerInstance!;
```

```
}
final WebSpeechTts _webSpeechTts = createWebSpeechTts();
String? _tempFilePath;
String? _currentLocalToken;
String? _lastLocalDoneToken;
int? _lastLocalDoneSession;
int? _lastLocalDoneQueuePos;
DateTime? _lastLocalDoneAt;
Timer? _localSpeakingPollTimer;
bool _localSpeakingPollInFlight = false;
int _localSpeakingPollSession = 0;
String? _localSpeakingPollToken;
bool _localSpeakingPollEverSpeaking = false;
DateTime? _localSpeakingPollStartedAt;
int _localSpeakingPollErrorCount = 0;

TencentTtsClient? _tencentTtsClient;
final Map<String, Uint8List> _onlineAudioCache = {};
final Map<String, Future<Uint8List>> _onlineAudioInFlight = {};
final int _onlinePrefetchDistance = 3;
bool _onlineTransitioning = false;

bool _initialized = false;
bool _playing = false;
bool _preparing = false;
bool _paused = false;
bool _endedNaturally = false;
int _session = 0;

String? _bookId;
int? _chapterIndex;
List<ReaderParagraph> _paragraphs = const [];

List<_ReadAloudChunk> _queue = const [];
int _queuePos = 0;

ReadAloudPosition? _position;

ReadAloudProvider();

bool _isPointsInsufficientError(Object e) {
  if (e is TencentCloudException) {
    if (e.code == 'PointsInsufficient') return true;
    if (e.code == 'HttpError') {
      final m = e.message;
      return m.contains('HTTP 402') ||
          m.contains('PointsInsufficient') ||
          m.contains('积分不足');
    }
    return e.message.contains('PointsInsufficient') ||
        e.message.contains('积分不足');
  }
  final s = e.toString();
  return s.contains('PointsInsufficient') ||
      s.contains('HTTP 402') ||
```

```
        s.contains('积分不足');
  }

  bool get playing => _playing;
  bool get preparing => _preparing;
  bool get paused => _paused;
  bool get endedNaturally => _endedNaturally;

  String? get bookId => _bookId;
  int? get chapterIndex => _chapterIndex;
  ReadAloudPosition? get position => _position;

  String? get highlightText =>
      (_playing || _preparing || _paused) ? _position?.highlightText : null;
  int? get highlightParagraphIndex =>
      (_playing || _preparing || _paused) ? _position?.paragraphIndex : null;

  void updateTranslationProvider(TranslationProvider tp) {
    if (identical(_tp, tp)) return;
    final old = _tp;
    _tp = tp;
    if (old != null) {
      try {
        old.removeListener(_onTtsConfigChanged);
      } catch (_) {}
    }
    _lastEngine = tp.readAloudEngine;

// ============================================================================
// 文件: ai/reading/qa_service.dart
// 原始行数: 364 | 保留行数: 160
// ============================================================================
import 'dart:async';
import 'dart:ui'; // for TextRange

import '../hunyuan/hunyuan_text_client.dart';
import '../local_llm/llm_client.dart';
import '../tencentcloud/tencent_credentials.dart';

String _getCurrentDateInfo() {
  final now = DateTime.now();
  final weekday = ['一', '二', '三', '四', '五', '六', '日'][now.weekday - 1];
  return '${now.year}年${now.month}月${now.day}日（周$weekday）';
}

class ReadingContextService {
  final Map<int, String> chapterContentCache;
  final int currentChapterIndex;
  final int currentPageInChapter;
  final Map<int, List<TextRange>> chapterPageRanges;

  ReadingContextService({
    required this.chapterContentCache,
    required this.currentChapterIndex,
    required this.currentPageInChapter,
```

```
    required this.chapterPageRanges,
  });

  String getCurrentChapterContent() {
    final chapterContent = chapterContentCache[currentChapterIndex] ?? '';
    if (chapterContent.isEmpty) return '';

   // 简单清理 XML 标签等
    final cleaned = _cleanContent(chapterContent);
    return cleaned;
  }

  String _cleanContent(String content) {
   // 简单移除 HTML/XML 标签
    return content.replaceAll(RegExp(r'<[^>]*>'), '').trim();
  }
}

// QA类型枚举
enum QAType {
  general,
  summary,
  keyPoints,
}

/// QA流式输出块
class QAStreamChunk {
  final String content;
  final String? reasoningContent;
  final bool isReasoning;
  final bool isComplete;

  QAStreamChunk({
    required this.content,
    this.reasoningContent,
    this.isReasoning = false,
    this.isComplete = false,
  });
}

String buildOnlineQaPrompt({
  required ReadingContextService contextService,
  required String question,
  required QAType qaType,
  String? history,
}) {
  String prompt;

  switch (qaType) {
    case QAType.summary:
      final content = contextService.getCurrentChapterContent().trim();
     prompt = '请总结以下内容，用清晰的要点列出：\n\n'
         '${content.isEmpty ? '（当前阅读内容为空）' : content}\n\n'
         '要求：仅基于内容总结，不要编造。\n'
         '输出：先列提纲（不超过6条），再给出总结（150字以内）。';
```

```
      break;
    case QAType.keyPoints:
      final content = contextService.getCurrentChapterContent().trim();
     prompt = '请从以下内容中提取关键要点，控制在5条以内：\n\n'
         '${content.isEmpty ? '（当前阅读内容为空）' : content}\n\n'
         '要求：仅基于内容提取，不要编造；覆盖事件、人物变化、伏笔线索。\n'
         '输出：不超过5条，每条一句话。';
      break;
    case QAType.general:
      prompt = _buildGeneralQaPrompt(
        question,
        contextService,
        history: history,
      );
      break;
  }

  return prompt;
}

String _buildGeneralQaPrompt(
  String userQuestion,
  ReadingContextService contextService, {
  String? history,
}) {
  final content = contextService.getCurrentChapterContent().trim();
  final historyText = (history ?? '').trim();
  final dateInfo = _getCurrentDateInfo();

  final buffer = StringBuffer()
    ..writeln('你是阅读助手。请基于【当前阅读内容】与【历史问答】作答，必要时可联网搜索补充信息。')
    ..writeln()
    ..writeln('当前日期：$dateInfo')
    ..writeln()
    ..writeln('要求：')
    ..writeln('1）优先在内容中定位答案并直接回答。')
    ..writeln('2）必要时引用原文短句作为依据（可简短摘录）。')
    ..writeln('3）不要编造；只有确实找不到再说"文中未提及/需要更多上下文"。')
    ..writeln(
      '4）【历史问答】可能包含错误或过时信息：仅用于理解代词指代、上下文与用户偏好；一旦与【当前阅读内容】冲突，以【当前阅读内容】为准
    ..writeln('5）涉及时间判断时，以当前日期为准。')
    ..writeln('6）如用户询问实时信息（如天气、新闻、汇率等），请联网搜索后回答。')
    ..writeln()
    ..writeln('【当前阅读内容】')
    ..writeln(content.isEmpty ? '（当前阅读内容为空）' : content)
    ..writeln();

  if (historyText.isNotEmpty) {
    buffer
      ..writeln('【历史问答（最近对话）】')
      ..writeln(historyText)
      ..writeln();
  }

  buffer
```

```
    ..writeln('【用户问题】')
     ..writeln(userQuestion)
     ..writeln()
    ..writeln('请给出清晰、准确的回答。');

  return buffer.toString().trim();
}

String buildLocalQaPrompt({
  required ReadingContextService contextService,
  required String question,
  required QAType qaType,
  String? history,
}) {
  String userPrompt;
  switch (qaType) {
    case QAType.summary:
      final content = contextService.getCurrentChapterContent();
      final cleanedContent = _tailText(
        _squashSpaces(content.isEmpty ? '（当前阅读内容为空）' : content), 1600);
     userPrompt = '请总结以下内容，用清晰的要点列出：\n\n'
          '$cleanedContent\n\n'
        '要求：仅基于内容总结，不要编造。\n'
        '输出：先列提纲（不超过6条），再给出总结（150字以内）。';
      break;
    case QAType.keyPoints:
      final content = contextService.getCurrentChapterContent();

// ================================================================================
// 文件: ai/illustration/illustration_service.dart
// 原始行数: 723 | 保留行数: 160
// ================================================================================
import 'dart:async';
import 'dart:io';
import 'package:flutter/foundation.dart';
import 'package:http/http.dart' as http;
import 'package:path/path.dart' as path;
import 'package:uuid/uuid.dart';

import '../hunyuan/hunyuan_image_client.dart';
import '../hunyuan/hunyuan_text_client.dart';
import '../tencentcloud/tencent_credentials.dart';
import 'illustration_item.dart';

typedef _AnchorRange = ({
  int anchorStart,
  int anchorEnd,
});

class IllustrationService {
  final HunyuanImageClient _imageClient;
  final HunyuanTextClient _textClient;
  final String _baseStoragePath;

  static const Duration _pollInterval = Duration(seconds: 3);
```

```
static const int _maxPollCount = 40;

IllustrationService({
  required TencentCredentials credentials,
  required String baseStoragePath,
})  : _imageClient = HunyuanImageClient(credentials: credentials),
      _textClient = HunyuanTextClient(credentials: credentials),
      _baseStoragePath = baseStoragePath;

Future<List<IllustrationItem>> generateIllustrations({
  required List<String> paragraphs,
  required String chapterTitle,
  required int count,
  required bool useLocalModel,
  Future<String> Function(String prompt)? run,
  bool thinking = false,
  String? debugName,
}) async {
  if (paragraphs.isEmpty) return const <IllustrationItem>[];

  int effectiveCount = count;
  if (effectiveCount <= 0) {
    int totalChars = 0;
    for (final p in paragraphs) {
      totalChars += p.length;
    }
    if (totalChars <= 1800) {
      effectiveCount = 4;
    } else if (totalChars <= 4500) {
      effectiveCount = 8;
    } else {
      effectiveCount = 12;
    }
  }
  effectiveCount = effectiveCount.clamp(1, 12);
  if (effectiveCount > paragraphs.length) {
    effectiveCount = paragraphs.length;
  }

  final runFn =
      run ?? ((p) => _runOnlineTextModel(p, thinking: thinking));

  if (useLocalModel) {
    return _generateLocal(
      paragraphs: paragraphs,
      count: effectiveCount,
      chapterTitle: chapterTitle,
      runFn: runFn,
      debugName: debugName,
    );
  } else {
    return _generateOnline(
      paragraphs: paragraphs,
      count: effectiveCount,
      chapterTitle: chapterTitle,
```

```
      runFn: runFn,
      debugName: debugName,
    );
  }
}

Future<List<IllustrationItem>> _generateLocal({
  required List<String> paragraphs,
  required int count,
  required String chapterTitle,
  required Future<String> Function(String prompt) runFn,
  String? debugName,
}) async {
  final anchors = _buildAnchorsByWeightedLength(
    paragraphs: paragraphs,
    count: count,
  );

  final items = <IllustrationItem>[];

  for (int i = 0; i < anchors.length; i++) {
    final anchor = anchors[i];
    final text = paragraphs
        .sublist(anchor.anchorStart, anchor.anchorEnd + 1)
        .map((p) => p.trim())
        .join('\n');
    final truncatedText = _normalizeForPrompt(text, maxLen: 600);

   final prompt = '你是插画提示词助手。请把文字改写为“可画出来的一张插画画面描述”（用于文生图）。\n'
      '要求：\n'
      '1）不新增剧情事实；不新增人物名/地名；不要出现任何文字/水印/字幕。\n'
      '2）输出一行，用“；”分隔：主体；动作；环境；光影；氛围。\n'
      '3）画面要具体可视，至少补充两项细节（服饰/表情/道具/景别/色调/构图任选其二），尽量精炼（建议<=120字）。\n'
      '4）只输出答案本身，不要输出<think>或解释；如模型会输出<answer>标签，只保留<answer>内这一行。\n'
      '示例：小纸人；轻轻摇晃；木床上；清晨阳光；温馨。\n'
        '\n'
      '章节标题：$chapterTitle\n'
      '序号：第${i + 1}张\n'
      '文字：\n$truncatedText';

    if (kDebugMode) {
      debugPrint(
        '[ILLUSTRATION] local.prompt idx=$i len=${prompt.length} debugName=$debugName',
      );
    }

    String desc = await runFn(prompt);
    desc = _normalizeModelPrompt(_stripModelNoise(desc));
    desc = desc.replaceAll(RegExp(r'^\d+[\.■\s]+'), '');
    desc = desc.replaceAll(RegExp(r'^["“]|["”]$'), '').trim();
   if (desc.isEmpty) desc = '主体；动作；环境；光影；氛围（请补充具体内容）';

    items.add(
      IllustrationItem(
        id: const Uuid().v4(),
```

```
          anchorStart: anchor.anchorStart,
          anchorEnd: anchor.anchorEnd,
        role: '插画',
        title: '第${i + 1}张',
          subject: '',
          action: '',
          setting: '',
          time: '',
          weather: '',
          shot: '',
          camera: '',
          lighting: '',
          mood: '',
          composition: '',
          prompt: desc,
          status: IllustrationStatus.promptReady,
          createdAt: DateTime.now(),
        ),
      );
    }
    return items;
  }

  Future<List<IllustrationItem>> _generateOnline({
    required List<String> paragraphs,
    required int count,
    required String chapterTitle,

// ================================================================================
// 文件: presentation/widgets/illustration_panel.dart
// 原始行数: 1399 | 保留行数: 180
// ================================================================================
import 'dart:async';
import 'dart:io';
import 'dart:math' as math;
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:shared_preferences/shared_preferences.dart';
import '../../ai/illustration/illustration_item.dart' as ill;
import '../../ai/tencentcloud/tencent_cloud_exception.dart';
import '../../ai/config/auth_service.dart';
import '../../ai/tencentcloud/embedded_public_hunyuan_credentials.dart';
import '../../ai/local_llm/model_manager.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_tokens.dart';
import '../models/ai_chat_model_choice.dart';
import '../providers/ai_model_provider.dart';
import '../providers/illustration_provider.dart';
import '../providers/translation_provider.dart';
import 'scene_image.dart';
import 'points_wallet.dart';
import 'ai_inference_top_row.dart';

class IllustrationPanel extends StatefulWidget {
```

```
  final bool isDark;
  final Color bgColor;
  final Color textColor;
  final String bookId;
  final Map<int, String> chapterTextCache;
  final int currentChapterIndex;
  final String? chapterIdSuffix;
  final ValueChanged<int>? onChapterIllustrationsGenerated;
  final void Function(String, {bool isError})? onShowTopMessage;

  const IllustrationPanel({
    super.key,
    required this.isDark,
    required this.bgColor,
    required this.textColor,
    required this.bookId,
    required this.chapterTextCache,
    required this.currentChapterIndex,
    this.chapterIdSuffix,
    this.onChapterIllustrationsGenerated,
    this.onShowTopMessage,
  });

  @override
  State<IllustrationPanel> createState() => _IllustrationPanelState();
}

class _IllustrationPanelState extends State<IllustrationPanel> {
  static const int _imageCostPoints = 20000;
  static const String _kIllustrationModelChoice =
      'illustration_model_choice_v1';
  static const String _kIllustrationThinkingEnabled =
      'illustration_thinking_enabled_v1';

  String _styleKey = '国风';
  String _ratioKey = '1:1';
  AiChatModelChoice _modelChoice = AiChatModelChoice.onlineHunyuan;
  bool _thinkingEnabled = true;
  final Set<String> _pendingGenerateIds = {};

  String _toastText = '';
  bool _toastIsError = true;
  Timer? _toastTimer;

  String _panelPopupText = '';
  Timer? _panelPopupTimer;

  static const Map<String, String> _stylePrompts = {
    '国风': '国风插画，细腻画风，电影感光影，无文字无水印',
    '水墨': '水墨国风插画，留白，柔和光影，无文字无水印',
    '厚涂': '厚涂插画，电影感光影，高细节，无文字无水印',
    '日漫': '日系插画，线条清晰，柔和光影，无文字无水印',
    '写实': '写实风格插画，电影级光影，高细节，无文字无水印',
  };
```

```
static const Map<String, String> _ratioToResolution = {
  '1:1': '1024:1024',
  '3:4': '768:1024',
  '4:3': '1024:768',
  '9:16': '768:1280',
  '16:9': '1280:768',
};

@override
void initState() {
  super.initState();
  unawaited(_loadPrefs());
}

Future<void> _loadPrefs() async {
  final prefs = await SharedPreferences.getInstance();

  String raw = (prefs.getString(_kIllustrationModelChoice) ?? '').trim();
  if (raw.isEmpty) {
    final legacyA =
        (prefs.getString('storybook_model_choice_v1') ?? '').trim();
    final legacyB = (prefs.getString('manga_model_choice_v1') ?? '').trim();
    final legacy = legacyA.isNotEmpty ? legacyA : legacyB;
    if (legacy.isNotEmpty) {
      raw = legacy;
      await prefs.setString(_kIllustrationModelChoice, legacy);
      await prefs.remove('storybook_model_choice_v1');
      await prefs.remove('manga_model_choice_v1');
    }
  }

  final choice =
      AiChatModelChoice.values.cast<AiChatModelChoice?>().firstWhere(
            (e) => e?.name == raw,
            orElse: () => null,
          );

  bool? thinking = prefs.getBool(_kIllustrationThinkingEnabled);
  if (thinking == null) {
    final legacyA = prefs.getBool('storybook_thinking_enabled_v1');
    final legacyB = prefs.getBool('manga_thinking_enabled_v1');
    final legacy = legacyA ?? legacyB;
    if (legacy != null) {
      thinking = legacy;
      await prefs.setBool(_kIllustrationThinkingEnabled, legacy);
      await prefs.remove('storybook_thinking_enabled_v1');
      await prefs.remove('manga_thinking_enabled_v1');
    }
  }

  final resolvedChoice = (!kIsWeb && Platform.isIOS &&
          (choice == AiChatModelChoice.localHunyuan18b ||
              choice == AiChatModelChoice.localMiniCpm05b))
      ? AiChatModelChoice.localHunyuan05b
      : choice;
```

```
    if (!mounted) return;
    final effectiveChoice = resolvedChoice ?? AiChatModelChoice.onlineHunyuan;
    setState(() {
      _modelChoice = effectiveChoice;
      _thinkingEnabled = thinking ?? true;
    });

   // 面板打开时，若在线模型且积分不足，提示登录
    if (effectiveChoice.isOnline) {
      WidgetsBinding.instance.addPostFrameCallback((_) {
        if (!mounted) return;
        final aiModel = context.read<AiModelProvider>();
        final tp = context.read<TranslationProvider>();
        final usingPersonal = tp.usingPersonalTencentKeys &&
            getEmbeddedPublicHunyuanCredentials().isUsable;
        if (!usingPersonal && aiModel.pointsBalance <= 0) {
        _showToast('积分不足，无法使用在线插画');
        }
      });
    }
  }

  Future<void> _setModelChoice(AiChatModelChoice value) async {
    if (_modelChoice == value) return;
    final aiModel = context.read<AiModelProvider>();
    final prevChoice = _modelChoice;
    if (value.isLocal) {
      final localModelId = switch (value) {
        AiChatModelChoice.localHunyuan05b => ModelManager.hunyuan_0_5b,
        AiChatModelChoice.localMiniCpm05b => ModelManager.minicpm4_0_5b,
        AiChatModelChoice.localHunyuan18b => ModelManager.hunyuan_1_8b,
        _ => ModelManager.hunyuan_1_8b,
      };
      if (aiModel.loaded && aiModel.activeLocalModelId != localModelId) {
        await aiModel.unloadLocalModel(reason: 'illustration_switch_local');
      }
    } else if (prevChoice.isLocal && aiModel.loaded) {
      await aiModel.unloadLocalModel(reason: 'illustration_switch_online');
    }
    setState(() => _modelChoice = value);
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_kIllustrationModelChoice, value.name);
  }

  Future<void> _setThinkingEnabled(bool value) async {
    if (_thinkingEnabled == value) return;

// ============================================================================
// 文件: presentation/providers/ai_model_provider.dart
// 原始行数: 540 | 保留行数: 100
// ============================================================================
import 'dart:async';
import 'dart:convert';
import 'package:flutter/foundation.dart';
```

```
import 'package:http/http.dart' as http;
import 'package:shared_preferences/shared_preferences.dart';
import '../../ai/local_llm/llm_client.dart';
import '../../ai/local_llm/model_manager.dart';
import '../../ai/local_llm/mnn_model_downloader.dart';
import '../../ai/local_llm/mnn_model_spec.dart';
import '../../ai/config/auth_service.dart';
import '../../ai/tencentcloud/tencent_api_client.dart';

enum ModelInstallStatus {
  notInstalled,
  installing,
  installed,
  failed,
}

class AiModelProvider extends ChangeNotifier {
  static const String _kPointsBalance = 'points_balance';
  static const String _kDebugPointsOverride = 'debug_points_override';
  static const String _kIllustrationCount = 'ai_illustration_count';
  static const String _kLegacyIllustrationCount = 'ai_storybook_page_count';
  static const String _kLastLocalModelId = 'ai_last_local_model_id';

  LlmClient? _llmClient;
  String _activeLocalModelId = ModelManager.defaultLocalModelId;
  int _pointsBalance = 0;
  int? _debugPointsOverride;
  int _illustrationCount = 0; // 0 = Auto
  bool _anyLocalModelInstalled = false;
  Timer? _localIdleUnloadTimer;
  DateTime? _lastLocalInferenceAt;
  int? _lastIdleScheduleAtMs;
  String? _lastIdleScheduleModelId;

  final Map<String, ModelInstallStatus> _installStatusByModelId = {};
  final Map<String, double> _downloadProgressByModelId = {};
  final Map<String, String> _currentDownloadFileByModelId = {};
  String? _downloadingModelId;
  MnnModelDownloader? _downloader;
  StreamSubscription? _statusSubscription;
  StreamSubscription? _progressSubscription;
  StreamSubscription? _fileSubscription;

  AiModelProvider() {
    TencentApiClient.onPointsBalanceChanged = (v) {
      if (_pointsBalance == v) return;
      unawaited(setPointsBalance(v));
    };
    _load();
  }

  bool get loaded => _llmClient != null && _llmClient!.isAvailable;
  String get activeLocalModelId => _activeLocalModelId;
  List<MnnModelSpec> get availableLocalModels => ModelManager.localModels;
```

```
  bool get anyLocalTextInstalled => _anyLocalModelInstalled;
  bool get localTextReady => loaded;

  int get illustrationCount => _illustrationCount;

  Future<void> setIllustrationCount(int value) async {
    final allowed = <int>{0, 4, 8, 12};
    final next = allowed.contains(value) ? value : 0;
    if (_illustrationCount == next) return;
    _illustrationCount = next;
    notifyListeners();
    final prefs = await SharedPreferences.getInstance();
    await prefs.setInt(_kIllustrationCount, next);
    if (prefs.containsKey(_kLegacyIllustrationCount)) {
      await prefs.remove(_kLegacyIllustrationCount);
    }
  }

  Future<void> _checkAnyLocalModelInstallation() async {
    final before = _anyLocalModelInstalled;
    try {
      for (final spec in ModelManager.localModels) {
        final ok = await ModelManager.isModelInstalled(spec.id);
        if (ok) {
          _anyLocalModelInstalled = true;
          if (before != _anyLocalModelInstalled) notifyListeners();
          return;
        }
      }
      _anyLocalModelInstalled = false;
      if (before != _anyLocalModelInstalled) notifyListeners();
    } catch (_) {
      _anyLocalModelInstalled = false;
      if (before != _anyLocalModelInstalled) notifyListeners();
    }
  }

  Future<String?> _findInstalledLocalModelId() async {
    if (await ModelManager.isModelInstalled(_activeLocalModelId)) {
      return _activeLocalModelId;
    }
    for (final id in ModelManager.preferredLocalModelIds) {

// ================================================================================
// 文件: ai/local_llm/llm_client.dart
// 原始行数: 272 | 保留行数: 50
// ================================================================================
import 'dart:async';
import 'dart:convert';
import 'dart:io';
import 'package:flutter/foundation.dart';
import 'package:path/path.dart' as p;
import 'mnn_client.dart';
import 'model_manager.dart';
```

```
abstract class LlmClient {
  bool get isAvailable;
  String? get currentModel;

  Future<bool> initialize({String? model});
  Future<String> generate({
    required String prompt,
    int maxTokens = 512,
    double temperature = 0.7,
    double topP = 0.9,
    int topK = 40,
    double minP = 0.0,
    double repetitionPenalty = 1.1,
  });

  Stream<String> generateStream({
    required String prompt,
    int maxTokens = 512,
    double temperature = 0.7,
    double topP = 0.9,
    int topK = 40,
    double minP = 0.0,
    double repetitionPenalty = 1.1,
  });

  Future<void> dispose();
}

class LlmClientMnn implements LlmClient {
  final MnnClient _client = MnnClient();
  String? _currentModel;

  void _debugLogPrompt(
    String where,
    String prompt, {
    required int maxTokens,
  }) {
    if (!kDebugMode) return;
    final trimmed = prompt.trimRight();
    const tailLen = 160;
    final tail = trimmed.length <= tailLen
        ? trimmed

// ================================================================================
// 文件: ai/local_llm/mnn_client.dart
// 原始行数: 256 | 保留行数: 70
// ================================================================================
import 'dart:async';
import 'dart:convert';
import 'dart:io';
import 'package:flutter/foundation.dart';
import 'package:flutter/services.dart';

class MnnClient {
  static const MethodChannel _channel = MethodChannel('airread/local_llm');
```

```
static const EventChannel _eventChannel =
    EventChannel('airread/local_llm_stream');

bool _isInitialized = false;
String? _modelPath;

MnnClient();

bool get isAvailable => _isInitialized;
String? get modelPath => _modelPath;

void _debugLogModelInput({
  required String where,
  required String prompt,
}) {
  if (!kDebugMode) return;
  final trimmed = prompt.trimRight();
  const tailLen = 160;
  final tail = trimmed.length <= tailLen
      ? trimmed
      : trimmed.substring(trimmed.length - tailLen);
  debugPrint(
    '[MnnClient][$where] modelPath=$_modelPath inLen=${trimmed.length} inTail=${jsonEncode(tail)}'
  );
}

static Future<bool> isPlatformAvailable() async {
 // 桌面端暂时返回 true 以支持开发测试（使用模拟实现或通过插件支持）
  if (!kIsWeb &&
      (Platform.isWindows || Platform.isMacOS || Platform.isLinux)) {
    return true;
  }
  try {
    final isAvailable = await _channel.invokeMethod<bool>('isAvailable');
    if (isAvailable == false && Platform.isAndroid) {
      final error = await _channel.invokeMethod<String>('getNativeLoadError');
      if (error != null) {
        debugPrint('[MnnClient] MNN not available due to: $error');
      }
    }
    return isAvailable ?? false;
  } catch (e) {
    debugPrint('[MnnClient] isPlatformAvailable error: $e');
    return false;
  }
}

Future<bool> initialize({required String modelPath}) async {
  try {
    final result = await _channel.invokeMethod<bool>('initialize', {
      'modelPath': modelPath,
    });

    if (result == true) {
      _isInitialized = true;
```

```
        _modelPath = modelPath;
        return true;
      } else {
        _isInitialized = false;
        return false;
      }
    } on PlatformException catch (e) {

// ================================================================================
// 文件: ai/hunyuan/hunyuan_text_client.dart
// 原始行数: 115 | 保留行数: 70
// ================================================================================
import '../tencentcloud/tencent_api_client.dart';
import '../tencentcloud/tencent_credentials.dart';
import '../tencentcloud/embedded_public_hunyuan_credentials.dart';

/// 流式输出返回的数据结构
class ChatStreamChunk {
  final String content;
  final String? reasoningContent;
  final bool isReasoning;
  final bool isComplete;

  ChatStreamChunk({
    required this.content,
    this.reasoningContent,
    this.isReasoning = false,
    this.isComplete = false,
  });
}

class HunyuanTextClient {
  static const String _host = 'hunyuan.tencentcloudapi.com';
  static const String _service = 'hunyuan';
  static const String _version = '2023-09-01';
  static const String _region = 'ap-guangzhou';

  final TencentApiClient _api;
  final TencentCredentials _credentials;

  HunyuanTextClient({
    TencentApiClient? api,
    required TencentCredentials credentials,
  })  : _api = api ?? TencentApiClient(),
        _credentials = credentials;

  /// Think 模型：支持推理链，API 调用名见官方文档
  static const String thinkModel = 'hunyuan-2.0-thinking-20251109';
  /// Instruct 模型：指令遵循、对话、文学创作
  static const String instructModel = 'hunyuan-2.0-instruct-20251111';

  @Deprecated('Use chatStream for better UX')
  Future<String> chatOnce({
    required String userText,
    String model = instructModel,
```

```
  }) async {
    final resp = await _api.postJson(
      host: _host,
      service: _service,
      action: 'ChatCompletions',
      version: _version,
      region: _region,
      secretId: _credentials.secretId,
      secretKey: _credentials.secretKey,
      useProxy: !usingPersonalTencentKeys(),
      payload: {
        'Model': model,
        'Stream': false,
        'Messages': [
          {'Role': 'user', 'Content': userText},
        ],
        'EnableEnhancement': true,
        'ForceSearchEnhancement': true,
      },
    );

    final choices = resp['Choices'];
    if (choices is List && choices.isNotEmpty) {
      final first = choices.first;
      if (first is Map) {
        final msg = first['Message'];
        if (msg is Map) {

// ============================================================================
// 文件: ai/hunyuan/hunyuan_image_client.dart
// 原始行数: 101 | 保留行数: 70
// ============================================================================
import 'dart:convert';
import '../tencentcloud/tencent_api_client.dart';
import '../tencentcloud/tencent_credentials.dart';
import '../tencentcloud/embedded_public_hunyuan_credentials.dart';

class HunyuanImageClient {
  static const String _host = 'aiart.tencentcloudapi.com';
  static const String _service = 'aiart';
  static const String _version = '2022-12-29';
  static const String _region = 'ap-guangzhou';

  final TencentApiClient _api;
  final TencentCredentials _credentials;

  HunyuanImageClient({
    TencentApiClient? api,
    required TencentCredentials credentials,
  })  : _api = api ?? TencentApiClient(),
        _credentials = credentials;

  String _truncateUtf8(String s, {required int maxBytes}) {
    if (maxBytes <= 0) return '';
    final raw = s.trim();
```

```
    final rawBytes = utf8.encode(raw);
    if (rawBytes.length <= maxBytes) return raw;

    final out = StringBuffer();
    int used = 0;
    for (final r in raw.runes) {
      final ch = String.fromCharCode(r);
      final b = utf8.encode(ch).length;
      if (used + b > maxBytes) break;
      out.write(ch);
      used += b;
    }
    return out.toString().trimRight();
  }

  /// 提交文生图任务
  /// 返回 JobId
  Future<String> submitTextToImageJob({
    required String prompt,
    String resolution = '1024:1024',
    int revise = 1,
    int styles =
        201, // 201: 日系动漫, 202: 3D, 203: 水墨, 204: 油画, ... (这里先给个默认值，后续可扩展)
  }) async {
    final resp = await _api.postJson(
      host: _host,
      service: _service,
      action: 'SubmitTextToImageJob',
      version: _version,
      region: _region,
      secretId: _credentials.secretId,
      secretKey: _credentials.secretKey,
      useProxy: !usingPersonalTencentKeys(),
      timeout: const Duration(seconds: 60),
      maxRetries: 120,
      payload: {
        'Prompt': _truncateUtf8(prompt, maxBytes: 1024),
        'Resolution': resolution,
        'Revise': revise,
       // 'Styles': [styles.toString()], // 可选，暂时不传，由 prompt 控制
      },
    );

    final jobId = resp['JobId'];
    if (jobId == null || jobId.toString().isEmpty) {
      throw Exception('Failed to get JobId from response: $resp');
    }

// ============================================================================
// 文件: presentation/widgets/ai_hud.dart
// 原始行数: 3875 | 保留行数: 90
// ============================================================================
import 'dart:async';
import 'dart:convert';
import 'dart:io';
```

```
import 'dart:math' as math;
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:provider/provider.dart';
import 'package:shared_preferences/shared_preferences.dart';

import '../../ai/local_llm/mnn_model_spec.dart';
import '../../ai/local_llm/model_manager.dart';
import '../../ai/reading/qa_service.dart';
export '../../ai/reading/qa_service.dart' show QAStreamChunk, QAType;
import '../../ai/tencentcloud/embedded_public_hunyuan_credentials.dart';
import '../../ai/tencentcloud/tencent_credentials.dart';
import '../../ai/translation/translation_types.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_tokens.dart';
import '../models/ai_chat_model_choice.dart';
import '../providers/ai_model_provider.dart';
import '../providers/translation_provider.dart';
import '../providers/qa_stream_provider.dart';
import 'glass_panel.dart';
import 'illustration_panel.dart';
import 'points_wallet.dart';
import 'ai_inference_top_row.dart';

// Re-export ModelInstallStatus for use in this file
export '../providers/ai_model_provider.dart' show ModelInstallStatus;

enum AiHudRoute {
  main,
  qa,
  illustration,
  tencentSettings,
}

/// AI companion bottom sheet with in-panel navigation.
///
/// Design goals:
/// - Continuous features use unified Switch toggles.
/// - Remove the top-right close icon; sheet dismissal is handled by outside tap / drag.
/// - Each row has its own settings entry (chevron) and row-tap navigation.
class AiHud extends StatefulWidget {
  final Color bgColor;
  final Color textColor;
  final AiHudRoute initialRoute;
  final String? initialQaText;
  final bool autoSendInitialQa;

  /// Feature enabled state (controls whether quick actions should appear).
  final bool translateEnabled;

  /// Active state (controls whether translation is applied to the reader content).
  final bool translateActive;

  final ValueChanged<bool>? onTranslateChanged;
```

```
final bool readAloudEnabled;
final ValueChanged<bool>? onReadAloudChanged;

/// QA scope data (per-book)
final String bookId;

/// Reading context for AI QA (use effective/plain text, not raw HTML)
final Map<int, String> chapterTextCache;
final int currentChapterIndex;
final int currentPageInChapter;
final Map<int, List<TextRange>> chapterPageRanges;
final String? illustrationChapterIdSuffix;
final ValueChanged<int>? onChapterIllustrationsGenerated;

final void Function(String, {bool isError})? onShowTopMessage;

const AiHud({
  super.key,
  this.bgColor = Colors.white,
  this.textColor = AppColors.deepSpace,
  this.initialRoute = AiHudRoute.main,
  this.initialQaText,
  this.autoSendInitialQa = false,
  required this.translateEnabled,
  this.translateActive = false,
  this.onTranslateChanged,
  required this.readAloudEnabled,
  this.onReadAloudChanged,
  required this.bookId,
  required this.chapterTextCache,
  required this.currentChapterIndex,
```